

Data Governance and Compliance in AI and Data Projects

Slide by slide speaking script and question preparation

Lizhan Hong · Orange Wholesale (OW / OINIS)

July 2026

Contents

Part 1. Speaking script (about 16 to 19 minutes at a normal pace)	2
Slide 1. Title	2
Slide 2. What I will cover	2
Slide 3. What the project touches, and what it does not	2
Slide 4. The four principles	3
Slide 5. Three layers of anonymisation	3
Slide 6. Synthetic data, and its one limitation	3
Slide 7. Three trust zones	4
Slide 8. Lab versus production, stated as invariants	4
Slide 9. The default state: nothing leaves	4
Slide 10. Two control points on the only path out	5
Slide 11. Audit and failure behaviour	5
Slide 12. Enforced in code, not only documented	5
Slide 13. Honest status, not aspirational status	5
Slide 14. Supply chain, model artefacts, accountability	6
Slide 15. Reproducibility as a governance control	6
Slide 16. Lifecycle, exit and incident handling	6
Slide 17. Request one: access to the internal model gateway	7
Slide 18. Request two: a scoped extract of labelled historical data	7
Slide 19. What I am asking you to decide	8
Part 2. Question preparation	8
Scope and classification	8
Anonymisation and synthetic data	9
External calls and export	9
The two requests	10
Process and standards	11

Part 1. Speaking script (about 16 to 19 minutes at a normal pace)

One block per slide. Bracketed markers are cues, do not read them.

To bring it down to about 12 minutes, cut slides 4, 14 and 15 and shorten slide 6 to its last paragraph. Slides 7, 10, 11, 17 and 18 should never be cut: they carry the scope answer, the export controls and the two requests.

Slide 1. Title

Thank you for the time. I would like to walk through how data is handled in my project, following the six topics on the agenda. I will then close with two access requests that I need in order to continue, and for each one I will state the boundaries I am proposing rather than asking for open access.

Slide 2. What I will cover

Six topics, then two requests. I want to flag the requests now so nothing feels like a surprise at the end. The first is access to the internal model gateway. The second is a scoped extract of labelled historical data. Both are lab scope, and I will come to the detail once the control framework is clear.

Slide 3. What the project touches, and what it does not

Let me start with the data itself, because it determines everything else.

The project works with three kinds of data. Network structural metadata, meaning topology, device configuration and routing state. Operational telemetry, meaning per interface counters, utilisation, errors and latency. And machine generated synthetic data produced inside the lab.

What it does not touch matters just as much. No customer traffic content. No subscriber data. No billing or contractual data. All the reasoning the system performs is over operational metadata.

On classification, everything in the project is treated as Orange Restricted by default, and nothing is downgraded without an explicit decision.

One point on personal data. Customer reachability inside the system is represented by prefix and interface identifiers, not by customer identities. Network identifiers can be sensitive depending on context, so rather than argue about their legal status, I take the cautious route: they are pseudonymised before anything reaches a model. I will come back to that control.

Slide 4. The four principles

Four principles guided the design, and each one maps to something concrete in the code rather than to an intention.

Data minimisation: the system carries only the fields the task needs. Purpose limitation: data pulled for evaluation is used for evaluation only. Least privilege: the agents have read only access to the twin and no handle on production at all. And offline by default: in its normal configuration the system makes no external call whatsoever.

That last one is worth pausing on. The default state of this system is not a reduced mode. It is fully functional with zero external calls.

Slide 5. Three layers of anonymisation

On anonymisation there are three distinct layers, and I want to be precise about which is which, because they are often conflated.

The first layer is the twin itself. The twin is a structural abstraction. It reproduces how the network behaves, that is topology, routing logic, configuration structure and load patterns. It does not carry the traffic the real network carries. So before any other control applies, the twin is already a large reduction in sensitivity.

The second layer is synthetic data, which I will cover on the next slide.

The third layer is pseudonymisation, and it applies specifically on the path toward a language model. Device names, interface identifiers, addresses and customer prefixes are replaced by stable pseudonyms drawn from the project's canonical identity map. The model sees router R1 and link L7, never the real names. The mapping stays inside the lab, which means I can map an answer back to real identifiers locally for display, without those identifiers ever leaving. Alongside it there is minimisation: only a whitelist of fields is forwarded, so raw configuration text is never sent, only the structure of a difference, for example that a metric changed from one value to another.

Slide 6. Synthetic data, and its one limitation

On the synthetic data specifically, since this was asked in the earlier security thread.

It is machine generated inside the lab by the traffic model and the fault injector. It is deterministic and reproducible from a fixed seed. It is not derived from real production traces. So when I say synthetic, I mean genuinely generated, not anonymised real data.

And I want to state its limitation myself, because it matters for one of my requests later. Synthetic data is excellent for reproducibility, but it only contains the events I generate. In my current evaluation window there are three threshold breaches. Three events is not enough to validate the alerting side of the system with any statistical confidence, and I say so in my reports.

Slide 7. Three trust zones

This is the diagram I would ask you to look at most closely, because it answers the scope question directly.

There are three zones, and data movement between them is one directional and controlled.

The production zone holds the configuration repository and the NUAR telemetry platform. The agents never enter this zone. There is no live agent connection to production. Two movements cross the boundary, and both are one way. Production configuration is read once, inside the lab, to generate the twin. And historical telemetry is used offline in the lab for training and validation.

The lab zone is where all agent computation happens: the twin, the synthetic data, detection, diagnosis and forecasting. Inside the lab there is a further restriction that I want to highlight. The agents read the live twin read only, and they run their counterfactual experiments on a sandbox copy, never on the live twin.

The third zone is the optional model layer. It sits behind an interface and it is not part of the default system.

Slide 8. Lab versus production, stated as invariants

If you take one slide from the scope discussion, it is this one. These properties hold in every configuration of the system.

The agents never open a connection to production. The twin is a copy, and it carries no customer traffic. The agents read the live twin read only and mutate only a sandbox. Any operation that would modify a device, such as fault injection or recovery, is gated behind an explicit authorisation flag, and that flag has only ever been used against the simulated twin.

So to answer the question as it was put to me earlier: both agents operate in lab scope. The twin is a lab copy generated from production configuration, not the production system.

Slide 9. The default state: nothing leaves

Now external calls, which I expect is the part you care about most.

The starting point is that the rule based diagnosis, the analytic performance model and the whole evaluation harness run locally, with zero external calls, and with no API key required. The system is fully operable with no external model at all.

I want to be clear that this is not a degraded fallback. It is the shipped default. Everything I have presented in my technical reports so far was produced in this state.

Slide 10. Two control points on the only path out

When a model is enabled, everything passes through two control points, and there is exactly one path out of the lab.

The first control point is pseudonymisation and minimisation, which I described earlier. Nothing reaches a model without passing through it.

The second control point is an egress gate around every outbound call. It classifies the destination as local, internally hosted, or external, and applies policy per class. It verifies that the payload actually went through redaction and rejects raw payloads. It scans for anything sensitive that remains, for example a non pseudonymised address, a raw configuration marker, or a credential like token, and blocks on a match. It enforces size and rate limits. An external destination requires an explicit authorisation flag which is off by default. A public or unknown destination is blocked by construction.

The reason these are two separate controls rather than one component is that I assume the first one can fail. The gate checks independently.

Slide 11. Audit and failure behaviour

Two properties of the gate that matter for your side of the question.

First, audit. Every attempt, allowed or blocked, is written to an append only log: timestamp, destination class, a hash of the content, the size, whether it was pseudonymised, the decision and the reason. The log records the hash, not the payload.

Second, failure behaviour. It is fail closed. If the gate blocks a call, the system falls back to the local rule based path and records the event. A blocked egress reduces capability. It never leaks, and it never fails open.

Slide 12. Enforced in code, not only documented

This is the slide I would like to be judged on.

The invariant that there is only one path out is enforced by a test, not by a paragraph in a document. There is a static check asserting that the model client is called in exactly one module. If anyone adds a direct call elsewhere, the test suite fails.

And there is a self test command that assembles a representative diagnosis context, runs it through redaction and the egress gate for each destination class, and prints what would be sent, what is blocked and why, along with the audit summary. That command exists so that a reviewer does not have to take my word for anything. I can run it now if that is useful.

Slide 13. Honest status, not aspirational status

My approach to compliance has been to keep three things aligned: the architecture as documented, the behaviour as implemented, and the claims made in a review like this

one. If they ever diverge, the documentation is what is wrong.

Concretely, the architecture document carries a status table that separates three things. What is implemented and running today. What exists as a control point that activates when the optional layer is enabled. And what is a deployment decision that belongs to security policy rather than to me.

I mention this because it is easy to present designed controls as if they were running. I have tried not to do that, and if you find a place where I have, I would like to know.

Slide 14. Supply chain, model artefacts, accountability

Three shorter points to close this topic.

On the software supply chain, the core dependencies are permissive licences. One optional component is copyleft and is isolated in an optional extra, not linked into the core distribution.

On model governance, which is specific to AI projects: a trained model is itself a data carrier. The model artefacts in this project are trained only on lab and synthetic data. No checkpoint has been trained on production configuration, so no artefact can carry production content. If that were ever to change, it would be a scope question to bring to you first.

On accountability, I would like to state it plainly. Implementation and evidence are my responsibility. Scope decisions belong to my supervisors. Validation of the controls belongs to security. I am not asking anyone to accept a control on my say so.

Slide 15. Reproducibility as a governance control

I want to present reproducibility as a governance property rather than as a technical detail, because that is what it is.

Every experiment runs on a virtual clock with fixed seeds and deterministic tie breaking. Results are written to a frozen table with a manifest recording the seed, the configuration and the data window. Every figure in my reports can be regenerated from that frozen table without rerunning any model.

In governance terms this means two things. Any number I present can be traced and re-derived by someone other than me. And if a result is ever disputed, we can tell whether the data changed or the code changed. That is an audit property, not just good practice.

Slide 16. Lifecycle, exit and incident handling

Three practical points that I expect you to ask about, so I will answer them first.

On lifecycle, any data extract used in the project stays in the lab, is not copied to a personal machine, and is deleted at the end of the internship.

On exit, what is handed over is the code, the documentation and the reproducible protocol, not data. Credentials are revoked at the same point. The protocol is written so that whoever continues can regenerate results from a fresh extract rather than inheriting mine.

On incidents, if I ever suspected that something had crossed the boundary, the procedure is to stop the run, preserve the audit log, and notify my supervisors and security immediately, rather than investigating quietly first.

Slide 17. Request one: access to the internal model gateway

That covers the six topics. I have two requests, and I would like to present each with its boundaries.

The first is access to the internal model gateway, llmproxy, from my work environment.

I want to be clear about why this is the request I am making. My architecture recommends an internally hosted model precisely because it avoids external egress and third party retention. That is the conservative option, not the permissive one. Today it is not available to me, because the client libraries cannot be installed or imported reliably in my environment. The practical consequence is that I am left with either no model at all, or an option that goes outside the group. I am asking for the compliant path.

What I would do with it is specific. The diagnosis agent currently runs on a deterministic rule based reasoner, which I intend to keep as a reference. I would connect a model behind the same interface, calling the same tools and producing the same structured report, and compare the two under identical conditions: diagnostic accuracy, number of tool calls, time to conclusion, whether the evidence chain holds, and whether the proposed cause survives counterfactual replay. Without the gateway that comparison cannot be made at all.

The boundaries: lab scope only, pseudonymised operational metadata only, every call through the egress gate, every call audited, and fail closed to the local reasoner.

Slide 18. Request two: a scoped extract of labelled historical data

The second request is a scoped, read only extract of labelled historical data from NUAR. By labelled I mean historical per link counters, together with incident timestamps and the recorded cause for those incidents.

There are two reasons, and they are different in nature.

The first is evaluation labels. To measure whether a diagnosis is correct, I need cases where the true cause is known. Today my labels come from faults I inject myself, which means I am grading the system on my own scenarios. Real recorded incidents with their causes would let me evaluate against reality.

The second is statistical power. As I said earlier, my current window contains three

threshold breaches. The alerting results cannot be validated on three events. Real history would provide enough of them to say something defensible.

The boundaries I propose: one zone rather than the whole network, a bounded time window, operational metadata only with no customer identity, pseudonymised on ingest, kept in the lab with no export, read only, and deleted at the end of the internship.

And if full access is judged too broad, there is a middle path that still works for me: an extract prepared or pre pseudonymised by the data owner, or an aggregated version. I would rather work with a narrower dataset than have no external validation at all.

Slide 19. What I am asking you to decide

To summarise, in one slide.

The controls are built. They live in the code and not only in the documentation, and they can be demonstrated rather than asserted. The default configuration sends nothing anywhere.

The two decisions I am asking for are: access to the internal gateway, and a scoped labelled extract. For both I have proposed boundaries, and I am happy to tighten them to whatever the guidelines require. If either request needs to go through a formal process, I would appreciate being pointed to it, and I will follow up in writing with the scope as agreed.

Thank you. I am glad to take questions.

Part 2. Question preparation

Scope and classification

Is the configuration used to build the twin considered production data? It originates in production, so I treat it as Orange Restricted. It is read once, inside the lab, to generate the twin. What the agents work on afterwards is the twin, which is a structural copy. There is no ongoing connection from the agents back to the production repository.

Does the project process personal data? The system reasons over operational metadata: counters, utilisation, routing and adjacency state, configuration differences. It does not process subscriber data or traffic content. Customer reachability is represented by prefix identifiers rather than customer identities. Because network identifiers can be sensitive depending on context, they are pseudonymised before anything reaches a model, so I do not rely on that classification argument as the only protection.

Which scope do the agents run in, lab or production? Both agents run in lab scope. The twin is a lab copy, and historical telemetry is used offline in the lab. No agent has a live connection to production. Any write operation is gated behind an explicit flag that has only ever been used against the simulated twin.

Is the twin itself a security risk, since it mirrors the real network? It is a structural mirror, so it deserves the same classification as configuration data, which is how I treat it. It reduces risk in one respect, since it carries no customer traffic, and it concentrates risk in another, since it describes structure in one place. That is why it lives in the lab under the same access control as the rest of the environment, and why I would not propose exporting it.

Anonymisation and synthetic data

Who generates the synthetic data, and is it derived from real traces? It is machine generated inside the lab by the traffic model and the fault injector, deterministic and reproducible from a fixed seed. It is not derived from real production traces. It is generated, not anonymised.

What exactly would leave the lab if a model is enabled? A minimised, pseudonymised context: incident symptoms, metric names, adjacency information, and the structure of a configuration difference. Not raw configuration text, not real device names, not real addresses or prefixes. The reverse mapping stays inside the lab.

What if the pseudonymisation has a bug and something slips through? That is exactly why the egress gate is a separate second control rather than part of the same component. The gate independently scans the payload for anything that still looks sensitive and blocks on a match, and it rejects payloads that never went through redaction. If it blocks, the system falls back to the local reasoner and records the event. The design assumes the first control can fail.

Is pseudonymisation reversible, and by whom? Reversible only inside the lab, by the process that holds the mapping, and only so that an answer can be displayed with real names locally. The mapping is not sent anywhere and is not persisted outside the run.

Could a trained model memorise sensitive configuration? The model artefacts here are trained only on lab and synthetic data, so no checkpoint has seen production configuration. If training on real data ever became necessary, that would be a scope decision to bring to you before doing it, not something I would do silently.

External calls and export

Will the model API ever be in contact with public forums or the open internet? No. In the default configuration there is no external call at all. When a model is used, the requested path is the internally hosted gateway. Public or unknown destinations are blocked by construction, and an external third party destination would require an explicit authorisation flag that is off by default.

Would a provider train on our data? That is precisely why I am asking for the internal gateway rather than an external service. On an internally hosted path the question does not arise. If an external endpoint were ever authorised, the requirement would be a no

retention and no training arrangement, and that is a contractual decision rather than mine.

How do we verify what is actually sent, rather than trusting the description? Two ways. The append only audit log records every attempt with destination class, content hash, size, whether it was pseudonymised, and the decision with its reason. And the self test command runs a representative context through the whole path and prints what would be sent and what is blocked. You can run it yourself, and I can run it now.

What is the blast radius if something goes wrong? In the default configuration nothing leaves, because nothing is sent. With the model layer enabled, the worst case is that pseudonymised operational metadata for a single incident reaches an internally hosted endpoint. There is no path by which customer traffic content or raw configuration can be sent: minimisation drops those fields before the gate, and the gate blocks on them independently.

Can you rate limit or cap what goes out? Yes, the gate enforces size and rate limits, and both are configurable. If you want a specific ceiling, tell me the number and I will set it as the default.

The two requests

Why do you need a model at all, given that the rule based reasoner works? The rule based reasoner is a genuine reference point and I intend to keep it. But it encodes the cases I anticipated. The open question is whether a model generalises better to cases I did not anticipate, and whether it does so without losing evidence quality. I cannot answer that without running the comparison, and I cannot run the comparison without the gateway.

Why llmproxy rather than an external API? Because it keeps everything inside the group boundary. Asking for the internal gateway is the conservative request. If the internal path stays unavailable, the remaining alternatives are worse from a security point of view, and I would rather not take them.

Why do you need real data if you already have a simulator? For two things the simulator cannot provide. Ground truth labels from real incidents, so the diagnosis is measured against reality rather than against my own injected scenarios. And statistical power: my current window contains three threshold breaches, which is not enough to validate the alerting side.

What is the minimum you need from NUAR? Historical per link counters for one zone over a bounded window, plus incident timestamps and the recorded cause. Operational metadata only. I do not need customer identities, traffic content, or configuration text.

Can you continue without this data? Yes. The project continues either way. What changes is what I can claim. Without real labels the evaluation stays inside the simulation, and I would have to state that limit explicitly in every report. With them, the

results become defensible in a production discussion. So it is a question of the strength of the conclusions, not of the project stopping.

Who else would have access, and where would the data live? It would stay inside the lab environment, not on a personal machine, under the same access control as the rest of the lab. I am not proposing to share it further, and any additional access would be a decision for the data owner.

What happens to the data when your internship ends? Extracts are deleted and credentials revoked. What is handed over is the code, the documentation and the reproducible protocol, not data. The protocol is written so that a successor can regenerate results from a fresh extract.

Could the incident cause data itself be sensitive, since it describes where the network is weak? Yes, and I think that is the right concern to raise about this request. It is why I am proposing one zone rather than the whole network, a bounded window, no export, and deletion at the end. If the recorded causes are considered too sensitive even under those conditions, an alternative that still helps me is a version where the cause is reduced to a category rather than a free text description.

Process and standards

How do we know the documentation matches the code? Because the key invariant is enforced by a test rather than by prose. A static check asserts that the model client is called in exactly one module, so an additional path out would fail the test suite. The architecture document also carries a status table separating what is implemented from what is designed.

What about open source licence compliance? Core dependencies are permissive. One optional component is copyleft and is isolated in an optional extra, not linked into the core, so it is not part of the default distribution.

Who is accountable for what? Implementation and evidence are mine. Scope decisions belong to my supervisors. Validation of the controls belongs to security. If you tell me a control is insufficient, I will change the implementation rather than defend the document.

Is there a formal approval process we should route these requests through? That is partly what I am asking. If there is a standard process for gateway access or for a data extract, I would like to be pointed to it, and I will submit the scope in writing as agreed here so there is a record of the boundaries.